

第10章 内部排序

10.1 概述

10.2 插入排序

10.3 交换排序

10.4 选择排序

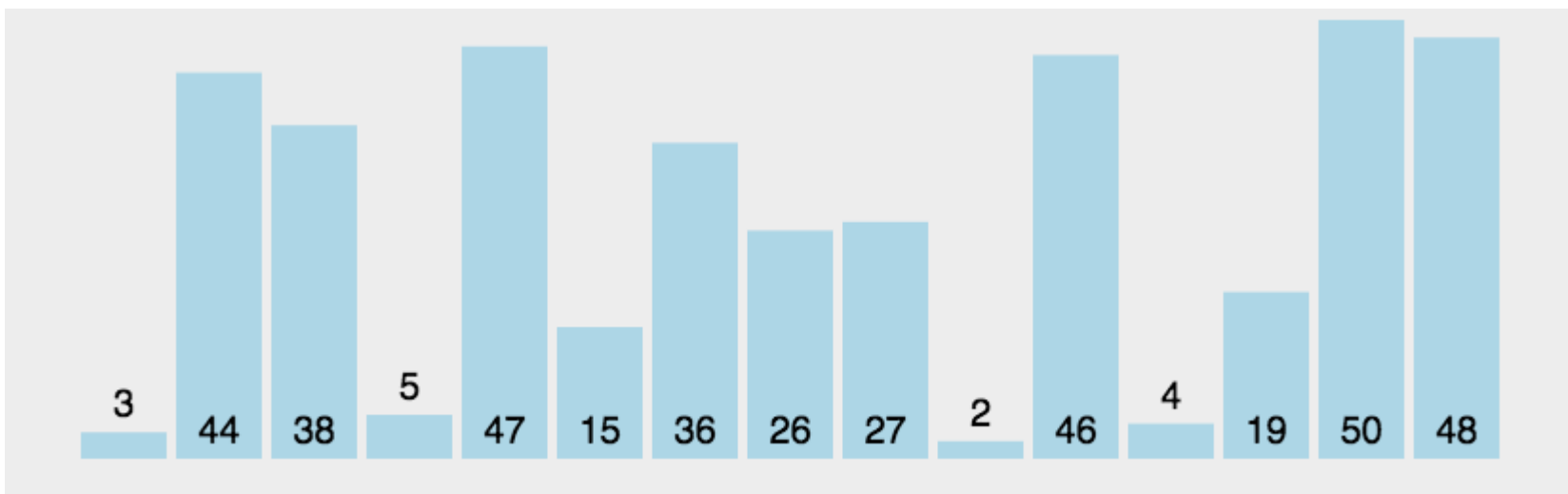
10.5 归并排序

10.6 基数排序

回顾

□ 冒泡排序

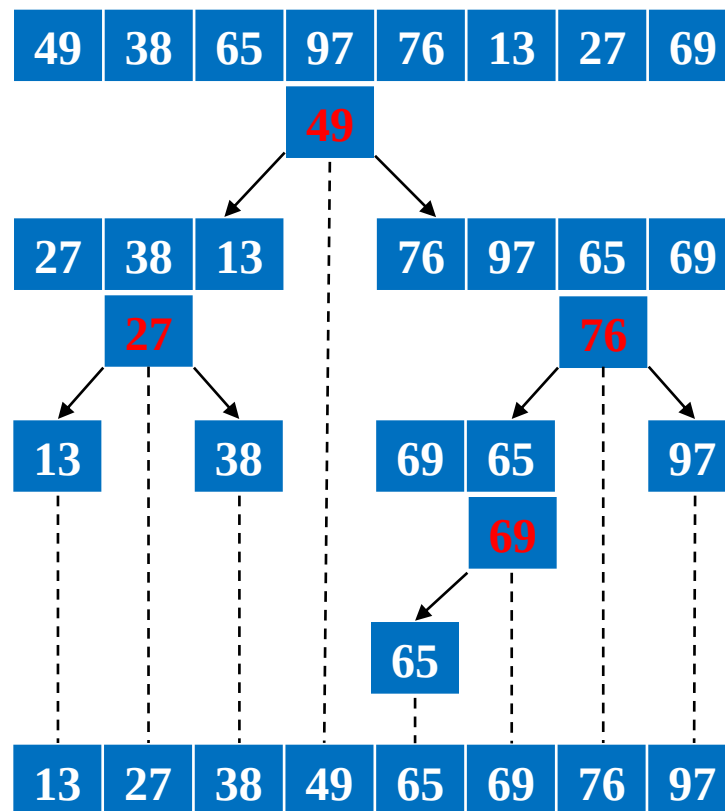
每一趟确定将一个数归位。即第一趟确定将末位上的数归位，第二趟将倒数第2位上的数归位，依次类推下去。如果有 n 个数进行排序，只需将 $n-1$ 个数归位，也就是要进行 $n-1$ 趟操作。



回顾

□ 快速排序

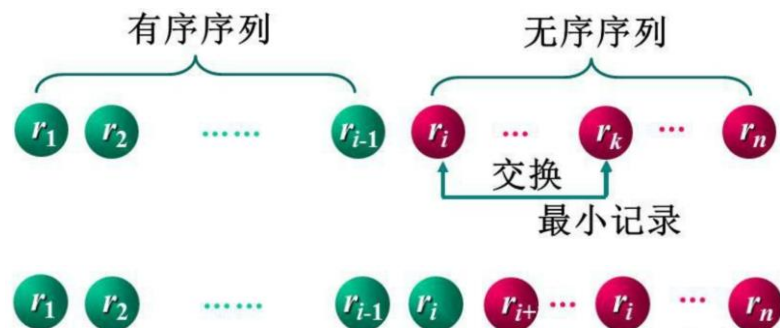
从待排序列中任取一个元素（例如取第一个）作为轴心(pivot)，所有比它小的元素一律前放，所有比它大的元素一律后放，形成左右两个子表；然后再对各子表重新选择轴心元素并依此规则调整，直到每个子表的元素只剩一个。此时便为有序序列了。



回顾

□ 简单选择排序

每经过一趟比较找出一个最小值，
与待排序列最前面的位置互换。



初始状态:

3	6	4	2	11	10	5
---	---	---	---	----	----	---

第1趟排序:

2	6	4	3	11	10	5
---	---	---	---	----	----	---

 (2在未排序序列中最小, 与3交换位置)

第2趟排序:

2	3	4	6	11	10	5
---	---	---	---	----	----	---

 (3是未排序序列中最小, 与6交换位置)

第3趟排序:

2	3	4	6	11	10	5
---	---	---	---	----	----	---

 (4是未排序序列中最小, 位置保持不变)

第4趟排序:

2	3	4	5	11	10	6
---	---	---	---	----	----	---

 (5是未排序序列中最小, 与6交换位置)

第5趟排序:

2	3	4	5	6	10	11
---	---	---	---	---	----	----

 (6是未排序序列中最小, 与11交换位置)

第6趟排序:

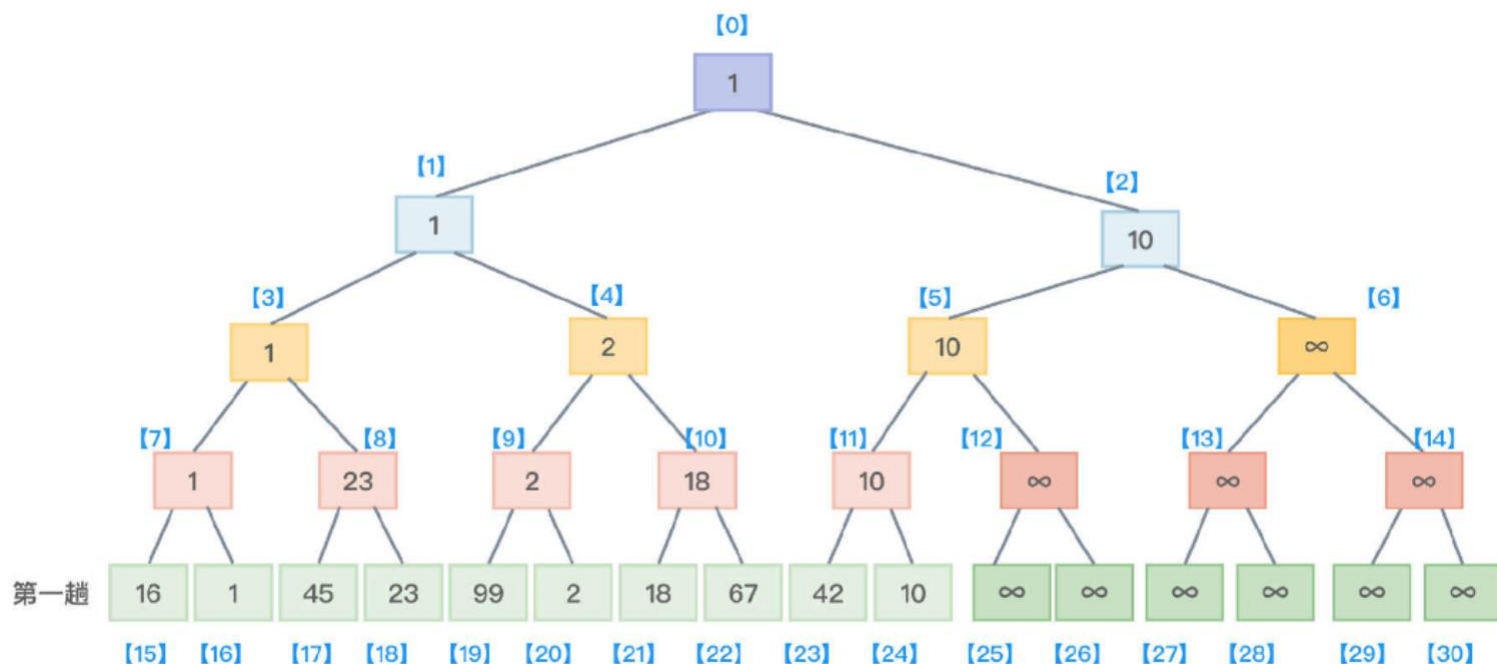
2	3	4	5	6	10	11
---	---	---	---	---	----	----

 (10是未排序序列中最小, 位置保持不变)

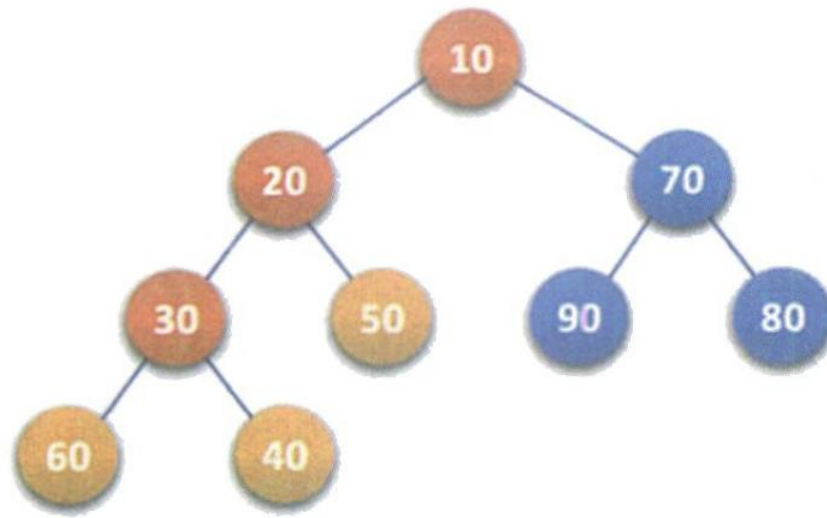
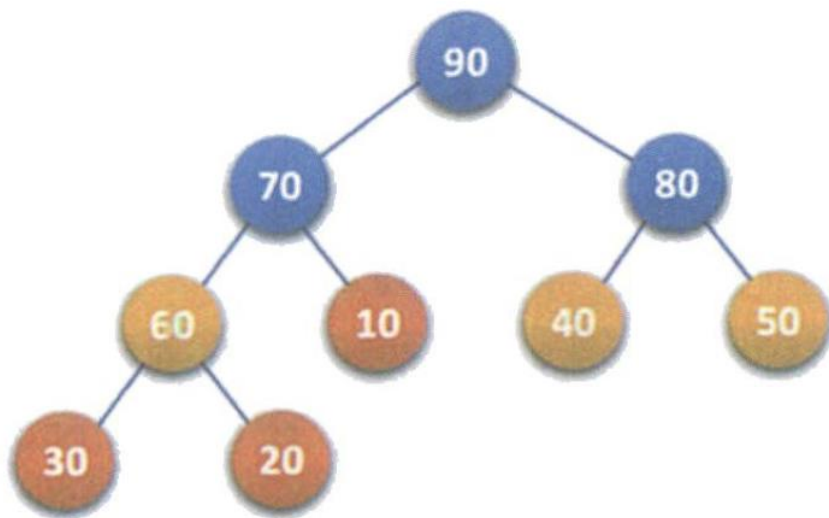
回顾

□ 树形选择排序

对 n 个记录的关键字进行两两比较，得到 $\lceil n/2 \rceil$ 个优胜者 (关键字小者)，作为第一步比较的结果保留下来。然后在这 $\lceil n/2 \rceil$ 个较小者之间再进行两两比较，...，如此重复，直到选出最小关键字的记录为止。



10.4 选择排序



10.4 选择排序

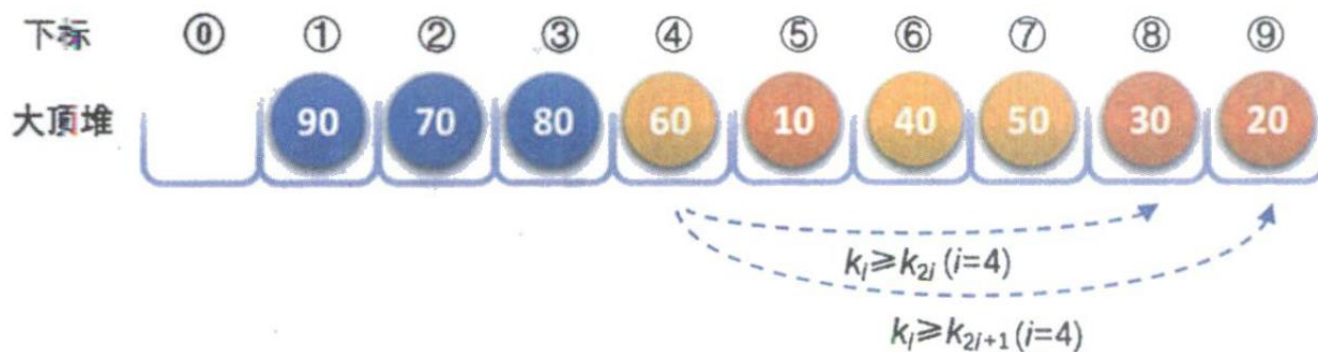
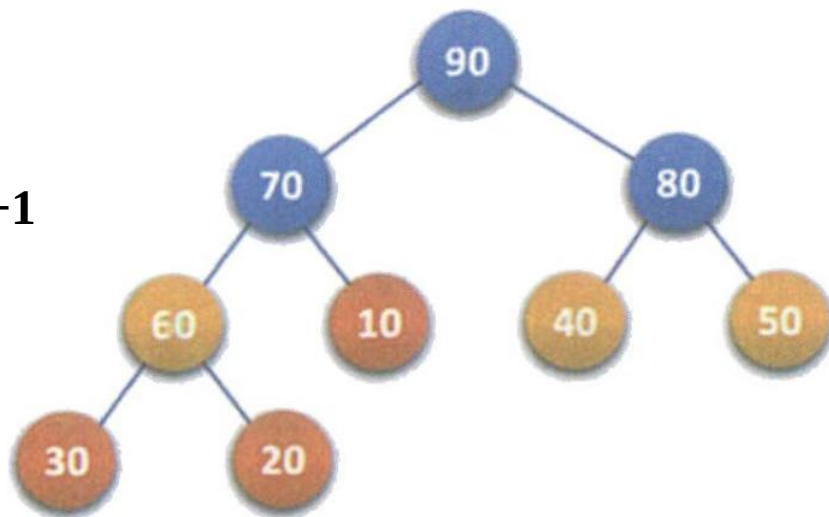
堆的定义：设有n个元素的序列 $k_1, k_2, \dots, k_n$ ，
当且仅当满足下述关系之一时，称之为堆。

$$\left\{ \begin{array}{l} k_i \leq k_{2i} \\ k_i \leq k_{2i+1} \end{array} \right. \quad \text{或者} \quad \left\{ \begin{array}{l} k_i \geq k_{2i} \\ k_i \geq k_{2i+1} \end{array} \right.$$

$$i=1, 2, \dots, n/2$$

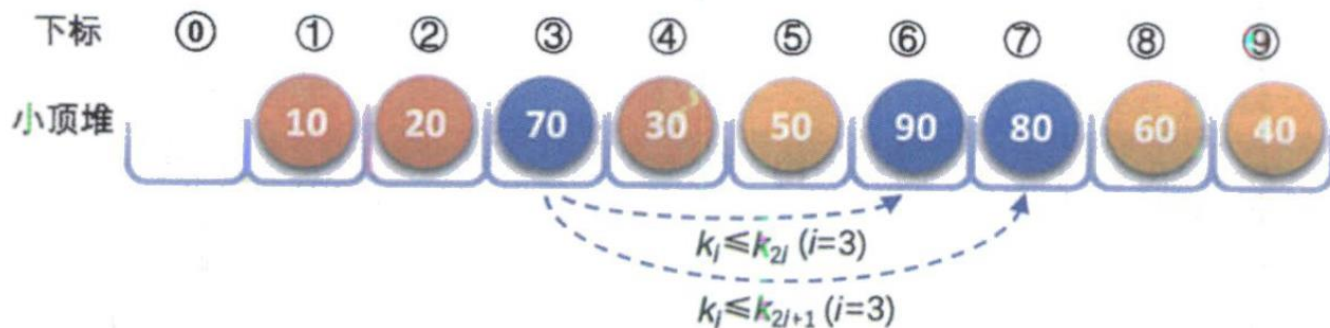
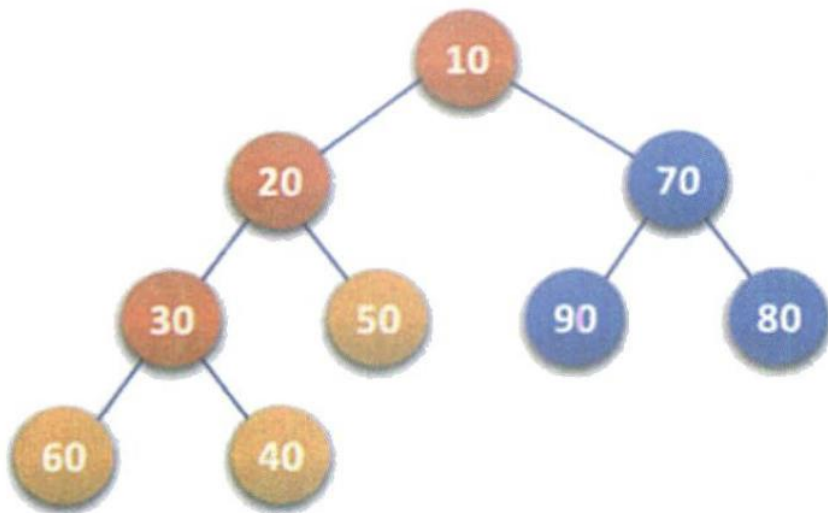
10.4 选择排序

$$\left\{ \begin{array}{l} k_i \geq k_{2i} \\ k_i \geq k_{2i+1} \end{array} \right.$$



10.4 选择排序

$$\left\{ \begin{array}{l} k_i \leq k_{2i} \\ k_i \leq k_{2i+1} \end{array} \right.$$



10.4 选择排序

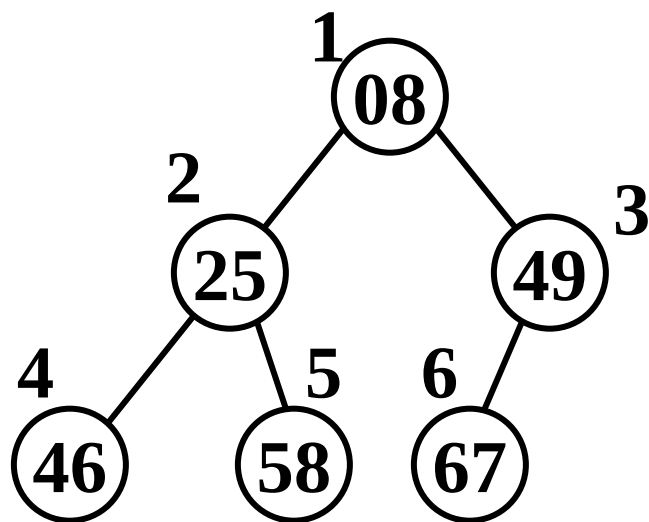
(1) 堆的定义

解释：如果让满足以上条件的元素序列 $(k_1, k_2, \dots, k_n)$ 顺次排成一棵**完全二叉树**，则此树的特点是：

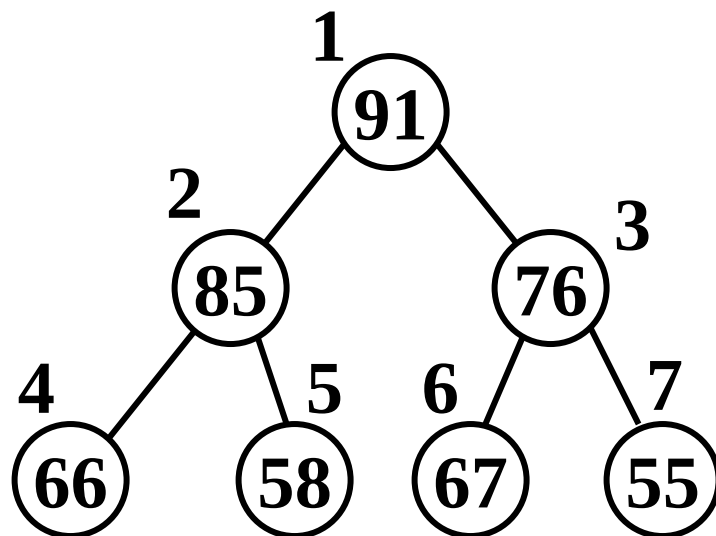
树中所有结点的值均大于（或小于）其左右孩子，此树的根结点（**即堆顶**）必最大（或最小）。

10.4 选择排序

例：有序列T1= (08, 25, 49, 46, 58, 67) 和序列T2= (91, 85, 76, 66, 58, 67, 55) , 判断它们是否“堆”？



(小顶堆)



(大顶堆)

10.4 选择排序

(2) 怎样建堆?

- 利用堆及其运算，可以很容易地实现选择排序的思路。
- 堆排序分为两个步骤：
 - 第一步，根据初始输入数据，利用堆的调整算法 ***HeapAdjust*** 形成初始堆；
 - 第二步，通过一系列的对象交换和重新调整堆进行排序。

10.4 选择排序

(2) 怎样建堆?

步骤：从最后一个**非终端结点**开始往前逐步调整，让每个双亲大于（或小于）子女，直到根结点为止。

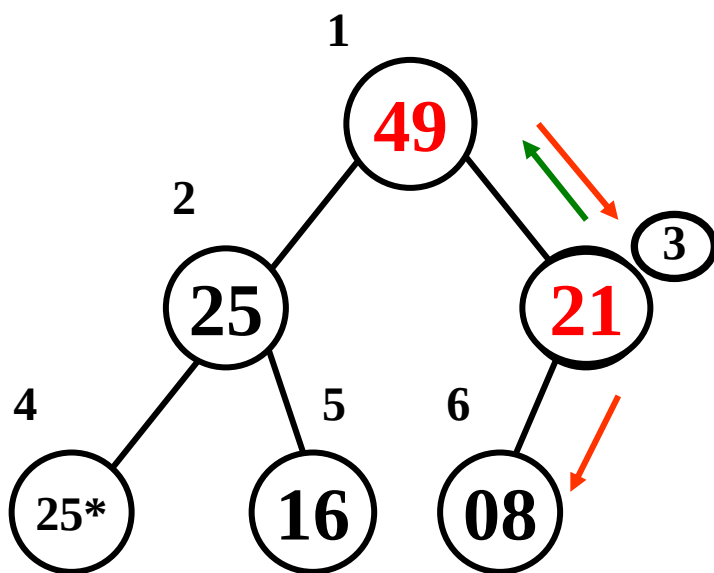
备注：终端结点（即叶子）没有任何子女，无需单独调整

例：关键字序列 $T = (21, 25, 49, 25^*, 16, 08)$ ，请建大根堆。

10.4 选择排序

(2) 怎样建堆?

为便于理解，先将原始序列画成完全二叉树的形式：
这样可以很清晰地从 $\lfloor n/2 \rfloor$ 开始调整。



(21, 25, 49, 25*, 16, 08)

完全二叉树的最后一个非终端结点编号必为 $\lfloor n/2 \rfloor$!! (性质5)

$i=3$: 49 大于 08, 不必调整;

$i=2$: 25 大于 25* 和 16, 也不必调整;

$i=1$: 21 小于 25 和 49, 要调整!

而且 21 还应当向下比较!!

10.4 选择排序

建堆算法 (等同于堆排序算法中的第一步)

参见教材P281-282

```
void HeapSort (HeapType &H ) {  
    //H是顺序表, 含有H.r[ ]和H.length两个分量  
    for ( i = length / 2; i > 0; - - i ) //把r[1...length]建成大根堆  
        HeapAdjust(r, i, length ); //使r[i...length]成为大根堆  
    .....  
} // HeapSort
```

*HeapAdjust*是针对结点*i*的堆调整函数, 其含义是: 从结点*i*开始到堆尾为止, 从上向下比较, 如果子女的值大于双亲结点的值, 则互相交换, 即把局部调整为大根堆。

10.4 选择排序

建堆算法 (等同于堆排序算法中的第一步)

针对结点 i 的堆调整函数 *HeapAdjust* 表述如下:

从结点 i 开始到当前堆尾 m 为止, 自上向下比较, 如果子女的值大于双亲结点的值, 则互相交换, 即把局部调整为大根堆。

10.4 选择排序

```
void HeapAdjust(int r[], int i, int m) { //将树（子树）根节点i调整为大顶堆
    //i为当前树根，m为二叉树最后一个结点
    current=i; temp=r[i]; //将树根临时存放，用于调整，temp暂存r[i]值
    child=2*i; //child是其左孩子
    //current为调整后的树（子树）根存放位置
    while(child<=m){ //检查是否到达当前堆尾，未到尾则整理
        if ( child<m && r[child].key<r[child+1].key )
            child= child+1; //让child指向两子女中的大者位置
        if ( temp.key>=r[child].key ) break; //根大则不必调整，函数结束
        else {
            r[current]=r[child]; //否则子女中的大者上移
            current= child; //current用于存放根节点元素位置
            child=2* child; //将根下降到子女位置并继续向下整理！
        }
    }
} // end while

r[current]=temp; // current用于存放树（子树）根节点的位置
} //直到自下而上都满足堆定义，再安置入口结点
```

10.4 选择排序

(3) 怎样进行整个序列的堆排序?

难点：将堆的当前顶点输出后，如何将剩余序列重新调整为堆？

方法：将当前顶点与堆尾记录交换，然后仿建堆动作重新调整，如此反复直至排序结束。

即：将任务转化为—>

$H.r[i \cdots m]$ 中除 $r[i]$ 外，其他都具有堆特征。

现调整 $r[i]$ 的值，使 $H.r[i \cdots m]$ 为堆。

10.4 选择排序

(3) 怎样进行整个序列的堆排序?

基于初始堆进行堆排序的算法步骤:

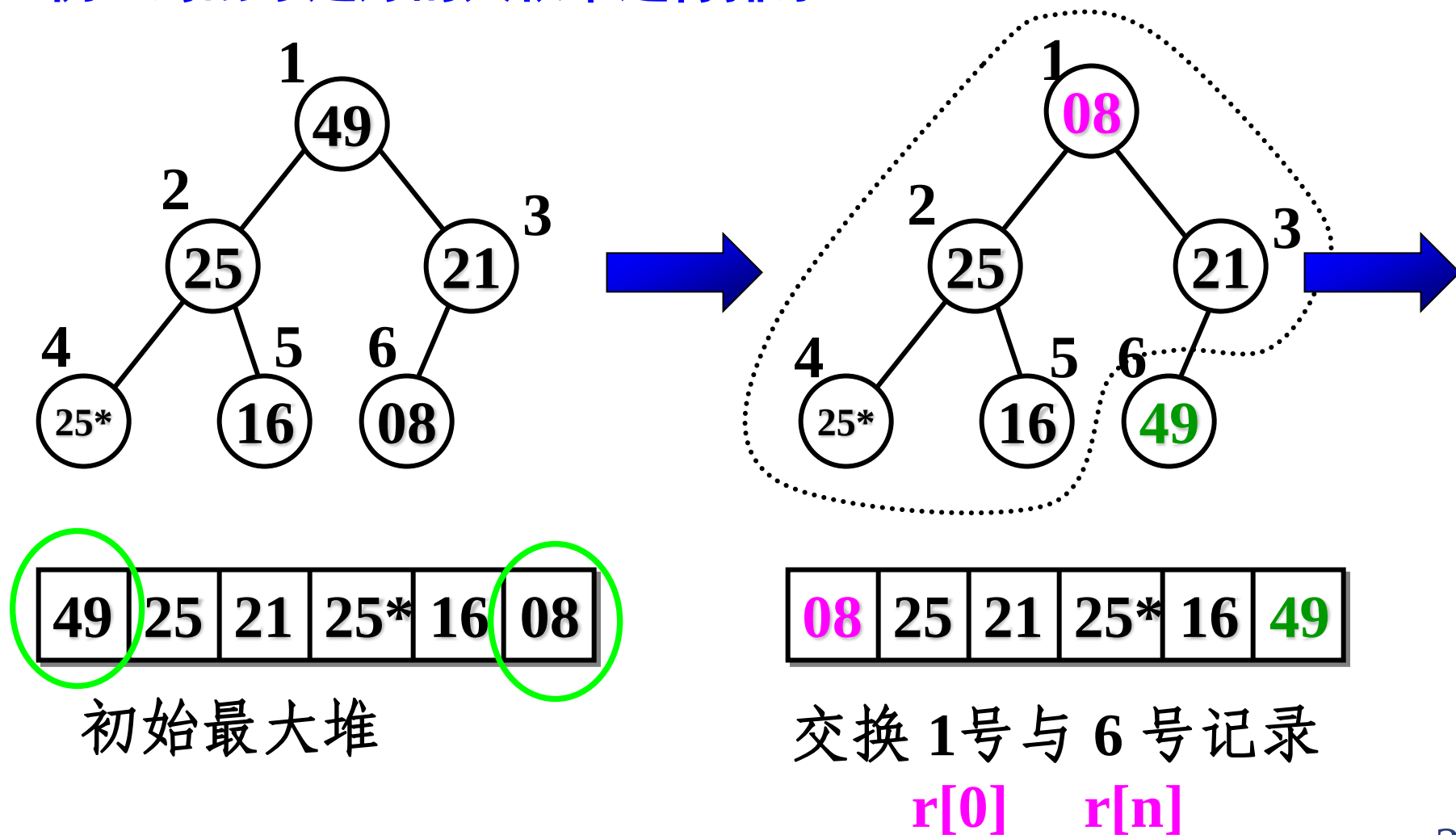
堆的第一个对象 $r[0]$ 具有最大的关键码, 将 $r[0]$ 与 $r[n]$ 对调, 把具有最大关键码的对象交换到最后;

再对前面的 $n-1$ 个对象, 使用堆的调整算法, 重新建立堆。结果具有次最大关键码的对象又上浮到堆顶, 即 $r[0]$ 位置;

再对调 $r[0]$ 和 $r[n-1]$, 然后对前 $n-2$ 个对象重新调整, ... 如此反复, 最后得到全部排序好的对象序列。

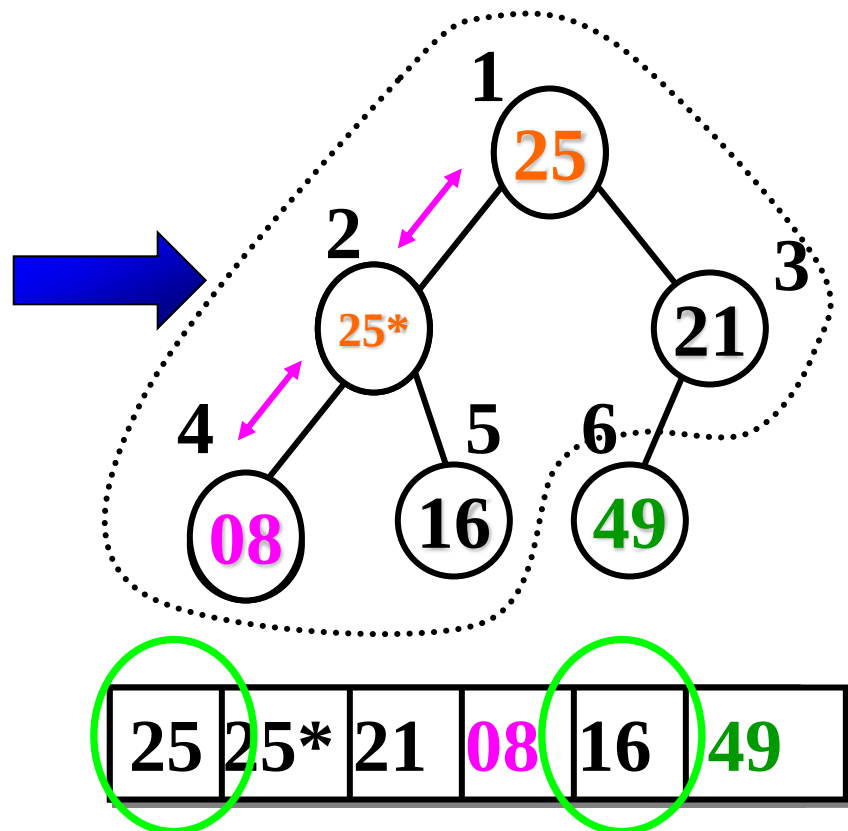
10.4 选择排序

例：对刚才建好的大根堆进行排序：

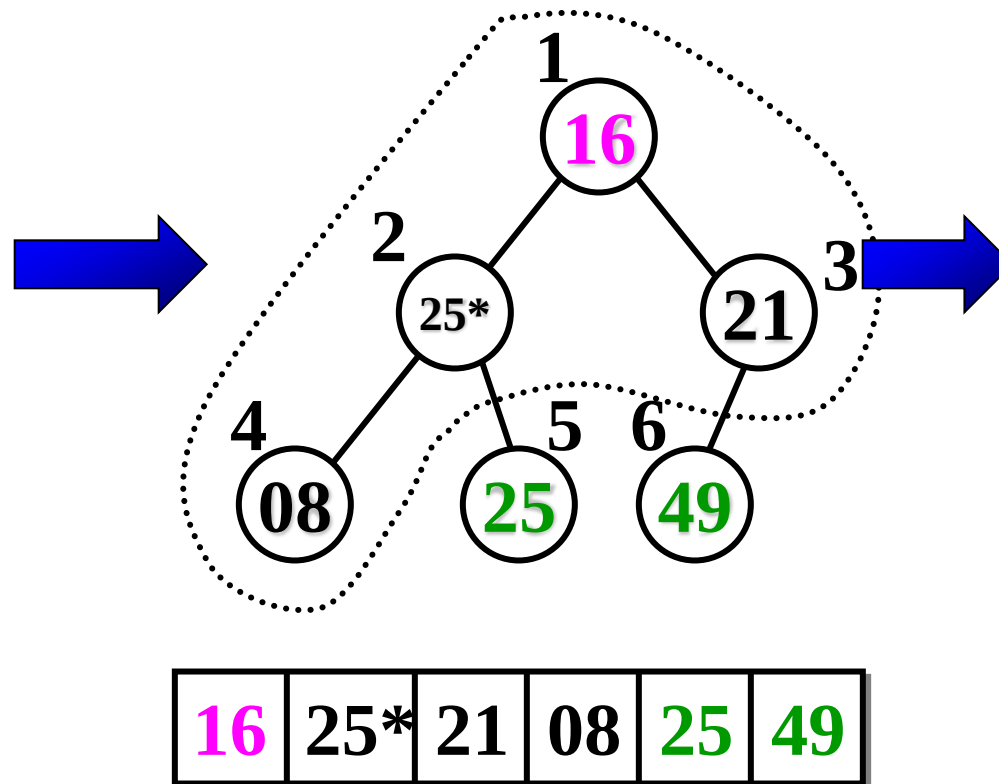


10.4 选择排序

例：对刚才建好的大根堆进行排序：



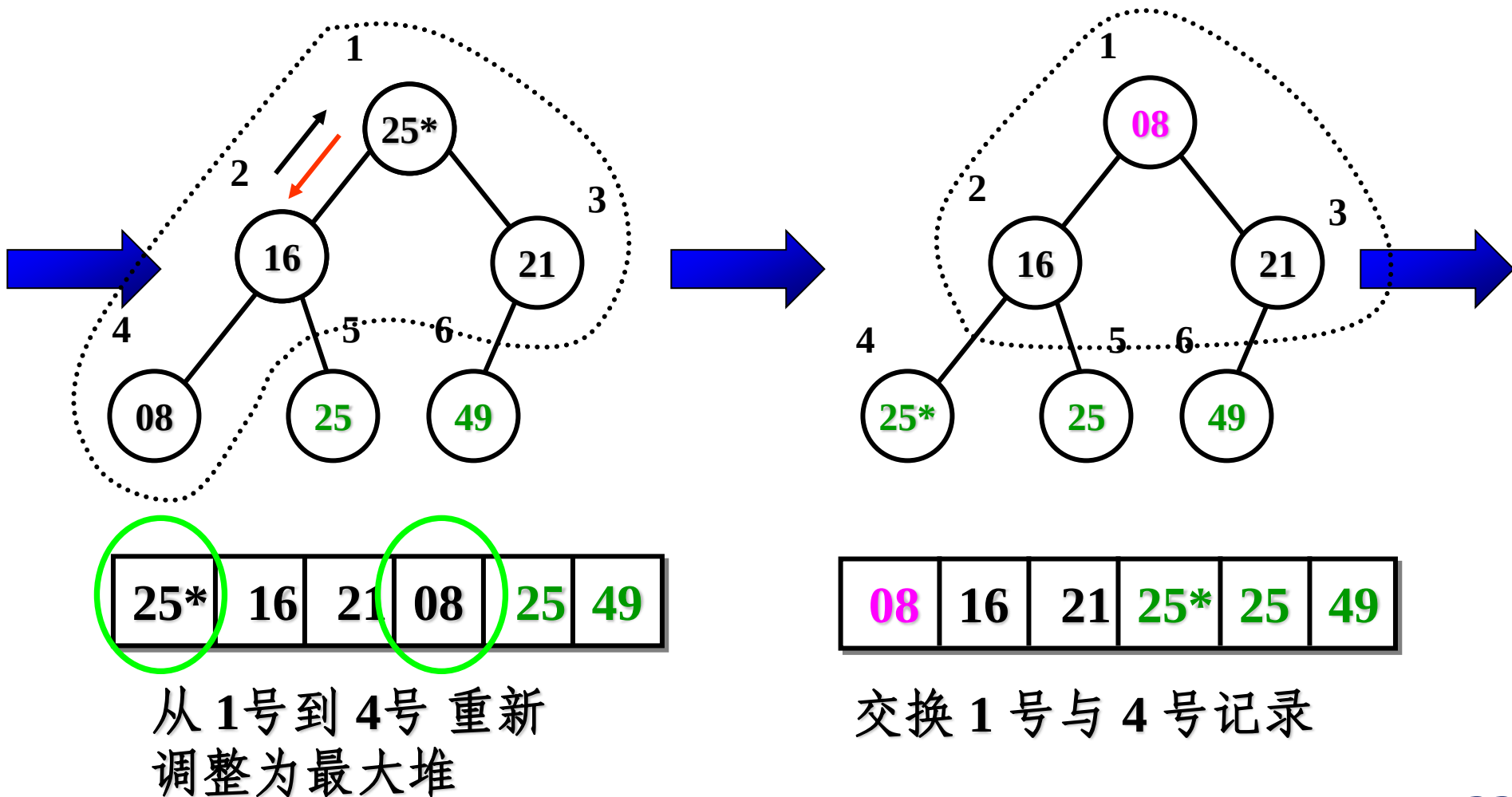
从 1 号到 5 号 重新
调整为最大堆



交换 1 号与 5 号记录

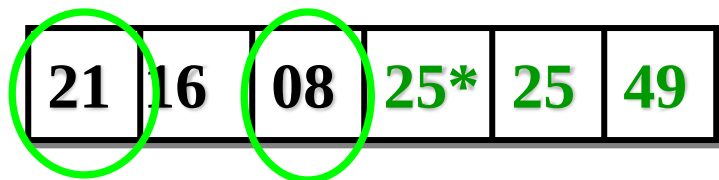
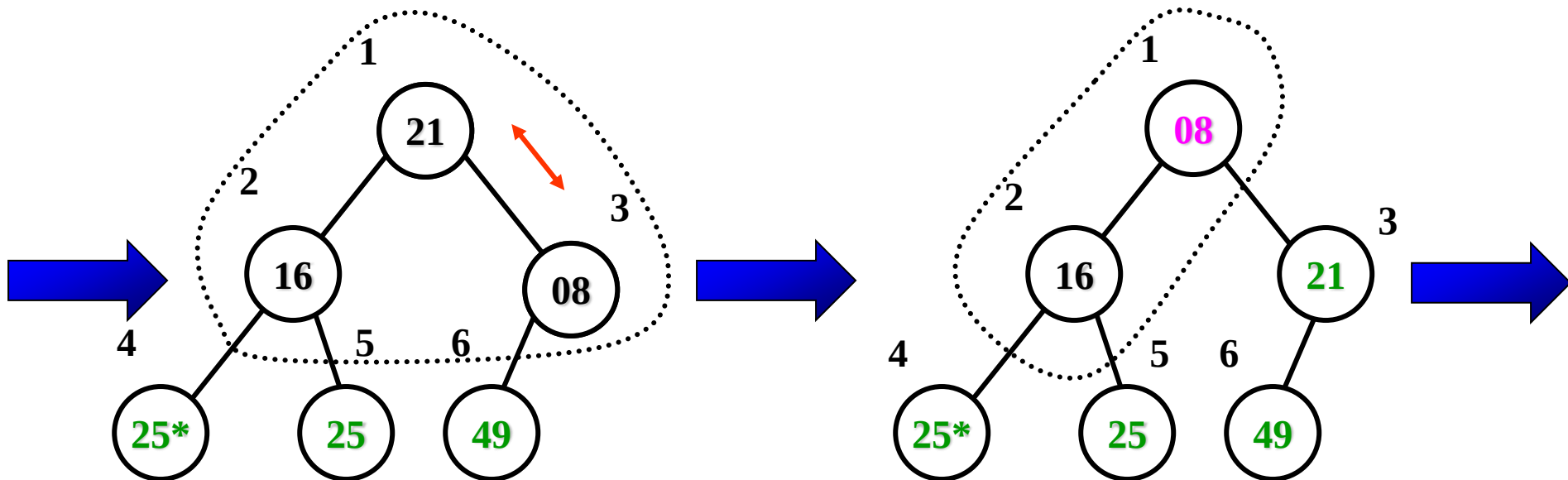
10.4 选择排序

例：对刚才建好的大根堆进行排序：

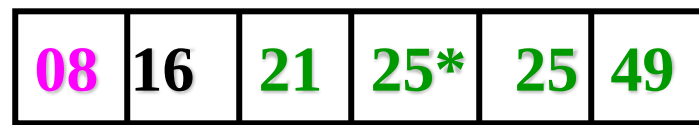


10.4 选择排序

例：对刚才建好的大根堆进行排序：



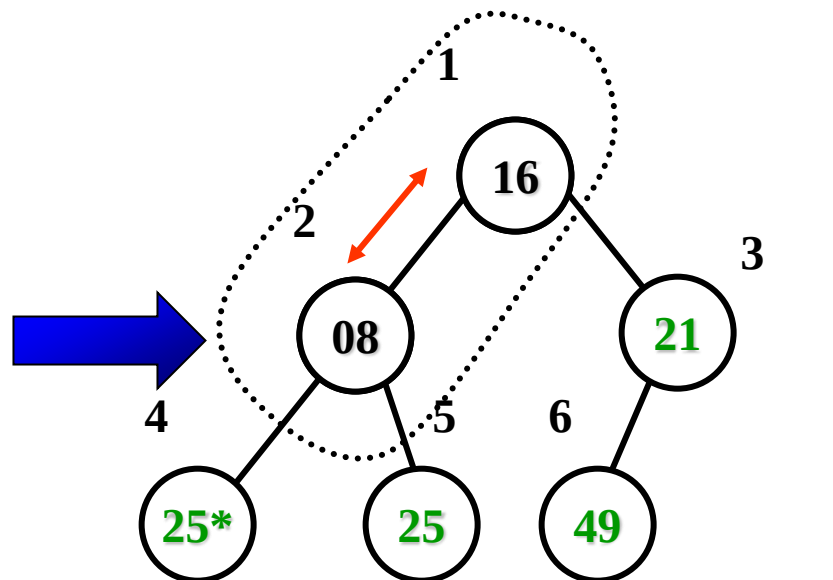
从 1 号到 3 号 重新
调整为最大堆



交换 1 号与 3 号记录

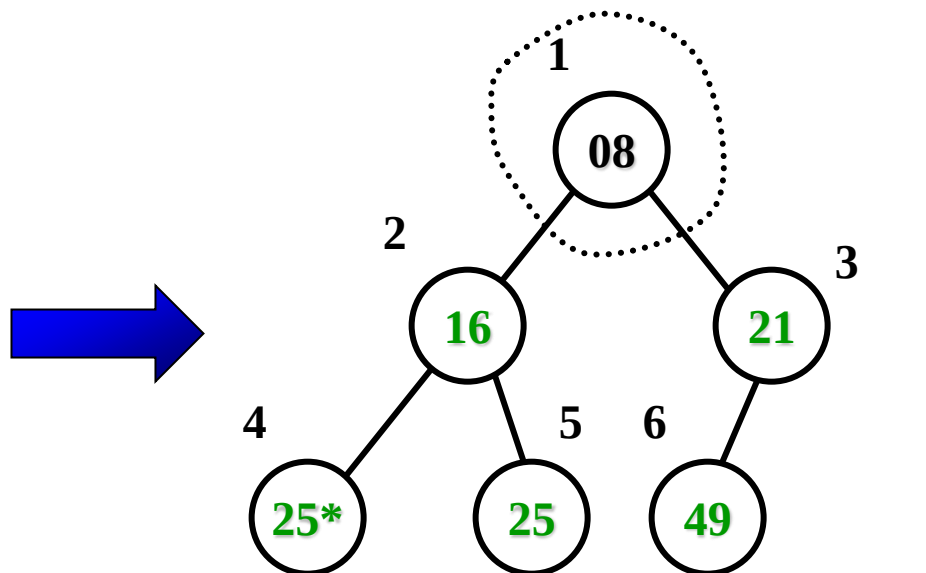
10.4 选择排序

例：对刚才建好的大根堆进行排序：



16	08	21	25*	25	49
----	----	----	-----	----	----

从 1 号到 2 号重新
调整为最大堆



08	16	21	25*	25	49
----	----	----	-----	----	----

交换 1 号与 2 号记录
排序完毕。

10.4 选择排序

堆排序的完整算法

参见教材P281-282

```
void HeapSort (HeapType &H ) {
```

```
//对顺序表H进行堆排序
```

```
    for ( i = H.length / 2; i > 0; -- i )
```

```
        HeapAdjust(H,i, H.length );    //for,建立初始堆
```

```
    for ( i = H.length; i > 1; --i) {
```

```
        H.r[1]  $\longleftrightarrow$  H.r[i];    //交换, 要借用temp
```

```
        HeapAdjust( H, 1,i-1 );    //重建最大堆, m=i-1
```

```
    }
```

```
}
```

因为每次从堆顶开始调整，故每次调用耗时 $O(\log_2 n)$

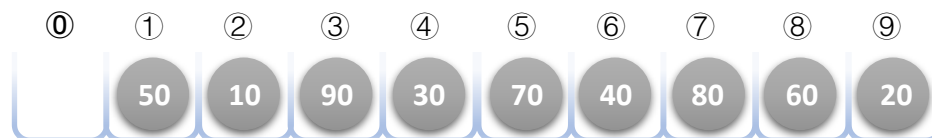
10.4 选择排序

关键代码

```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}

void HeapAdjust(SqList *L,int s,int m)
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

数据变化



10.4 选择排序

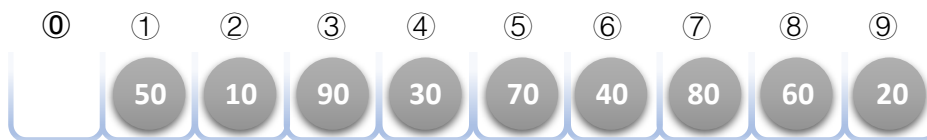
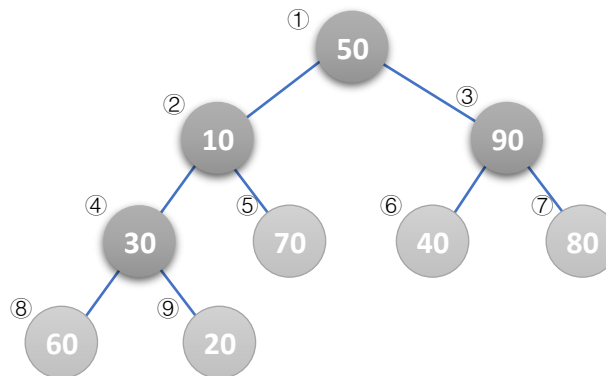
关键代码

```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}
```

```
void HeapAdjust(SqList *L,int s,int m)  s=4, m=9, j=8, temp=30
```

```
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

数据变化



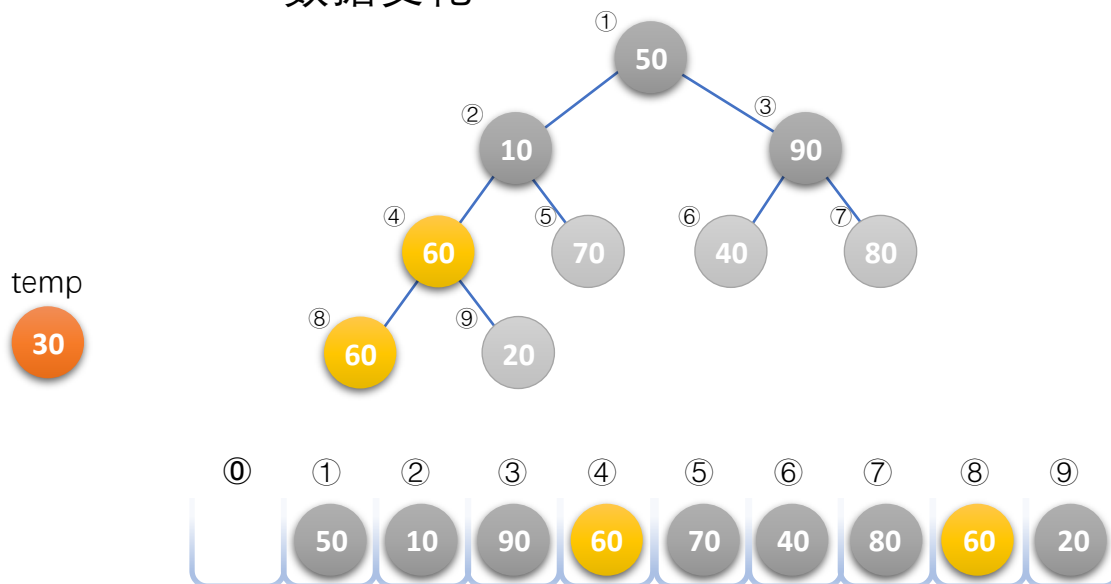
10.4 选择排序

关键代码

```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}

void HeapAdjust(SqList *L,int s,int m)
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

数据变化



s=4, j=8, m=9, temp=30, L->r[j]=60, L->r[j+1]=20, L->r[s]=30

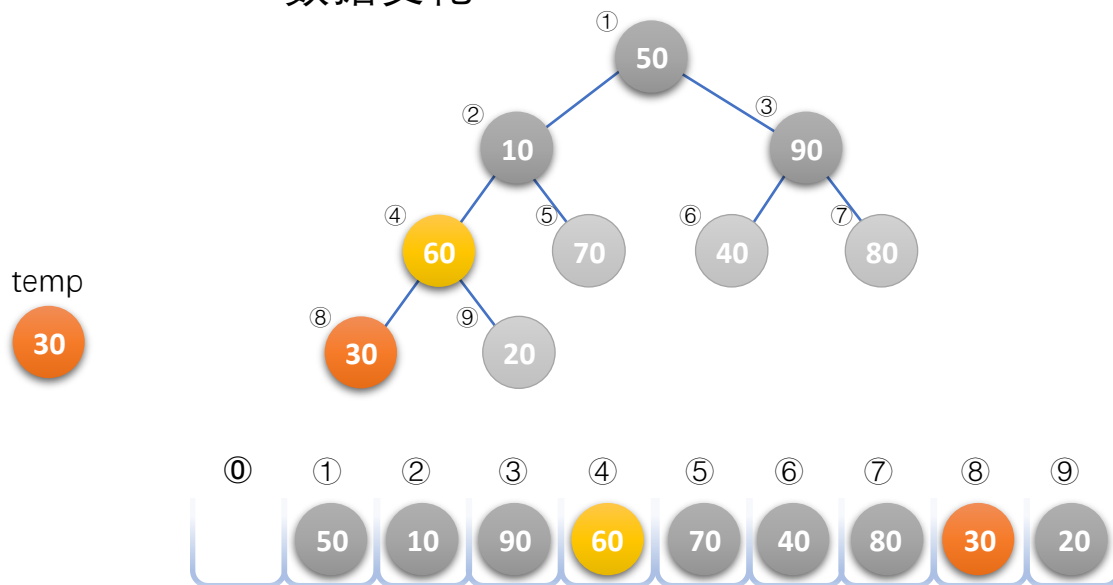
10.4 选择排序

关键代码

```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}

void HeapAdjust(SqList *L,int s,int m)
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

数据变化



s=8, j=8, temp=30, L->r[s]=30

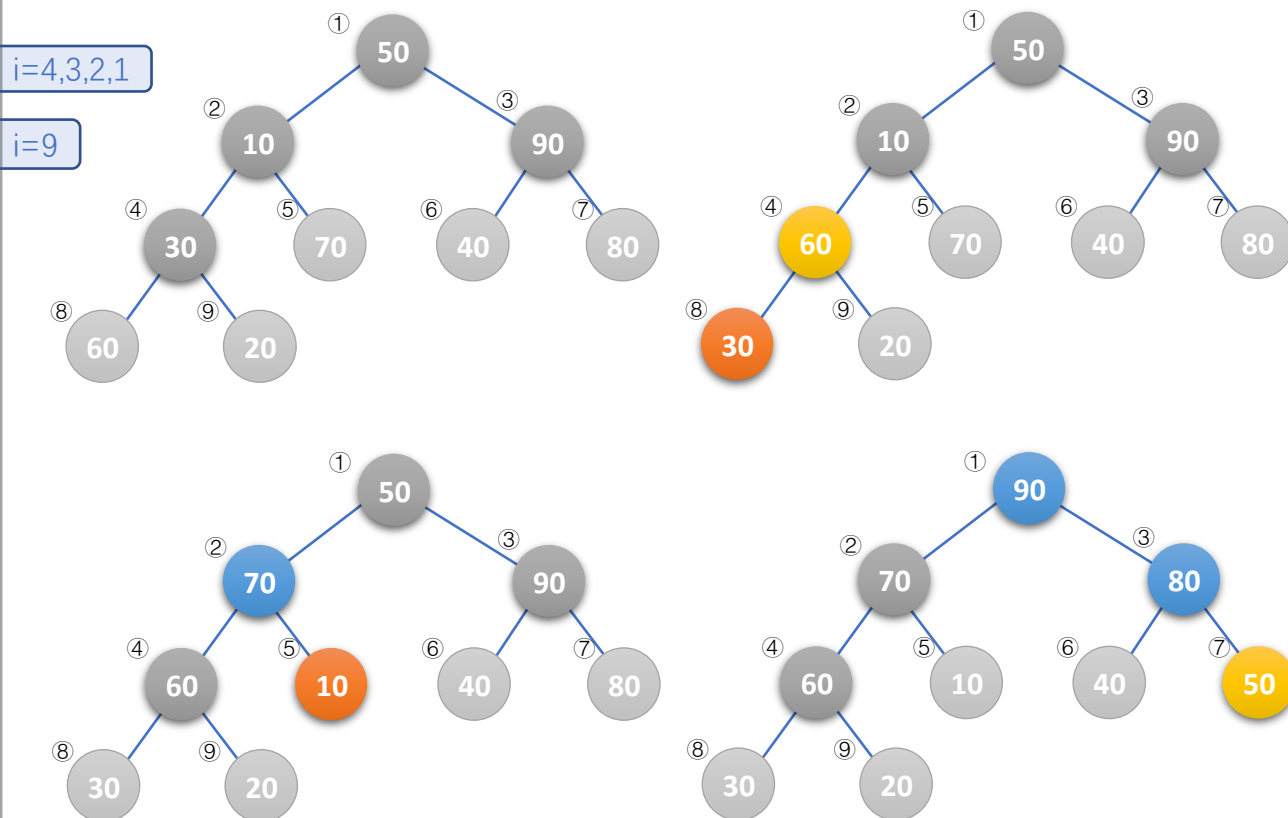
10.4 选择排序

关键代码

```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}

void HeapAdjust(SqList *L,int s,int m)
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

数据变化



10.4 选择排序

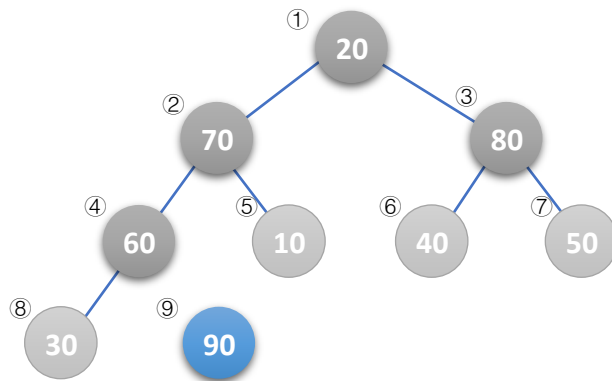
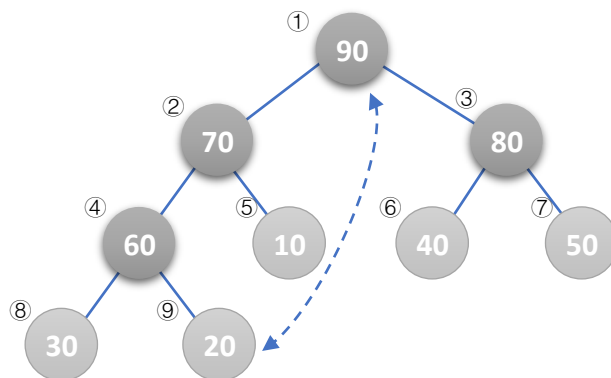
关键代码

```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}

void HeapAdjust(SqList *L,int s,int m)
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

i=9

数据变化



10.4 选择排序

关键代码

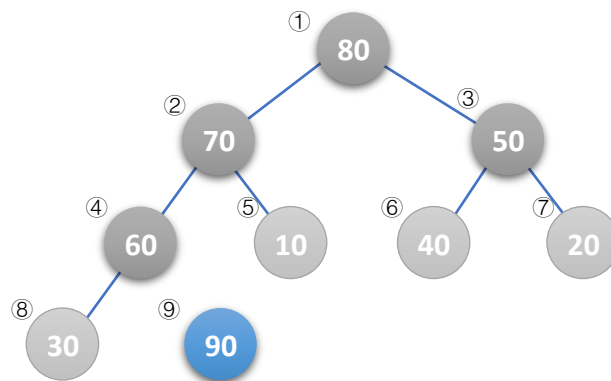
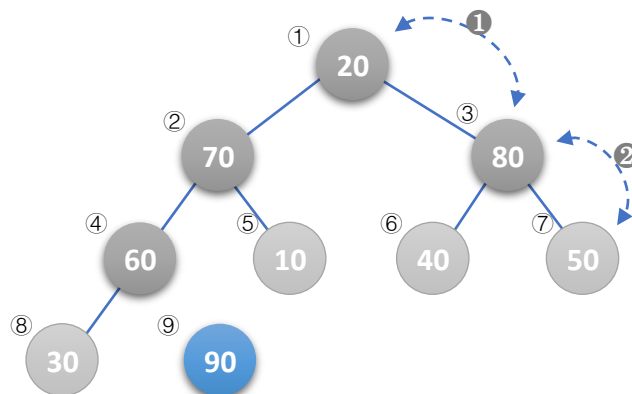
```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}
```

```
void HeapAdjust(SqList *L,int s,int m)
```

s=1, m=8

```
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

数据变化



10.4 选择排序

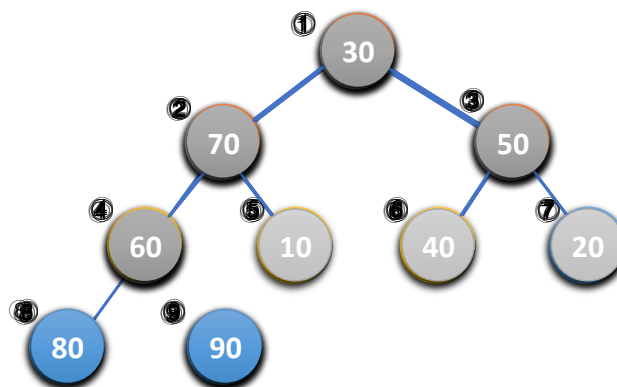
关键代码

```
void HeapSort(SqList *L)
{
    int i;
    for(i=L->length/2;i>0;i--)
        HeapAdjust(L,i,L->length);
    for(i=L->length;i>1;i--)
    {
        swap(L,1,i);
        HeapAdjust(L,1,i-1);
    }
}

void HeapAdjust(SqList *L,int s,int m)
{
    int temp,j;
    temp=L->r[s];
    for(j=2*s;j<=m;j*=2)
    {
        if(j<m && L->r[j]<L->r[j+1])
            ++j;
        if(temp>=L->r[j])
            break;
        L->r[s]=L->r[j];
        s=j;
    }
    L->r[s]=temp;
}
```

i=9,8,7,6,5,4,3,2

数据变化



10.4 选择排序

堆排序算法分析

- 时间效率： $O(n\log_2 n)$ 。因为整个排序过程中需要调用 $n-1$ 次 *HeapAdjust()* 算法，而算法本身耗时为 $\log_2 n$ ；
- 空间效率： $O(1)$ 。仅在第二个for循环中交换记录时用到一个临时变量temp。
- 稳定性： 不稳定。
- 优点： 对小文件效果不明显，但对大文件有效。

第10章 内部排序

10.1 概述

10.2 插入排序

10.3 交换排序

10.4 选择排序

10.5 归并排序

10.6 基数排序

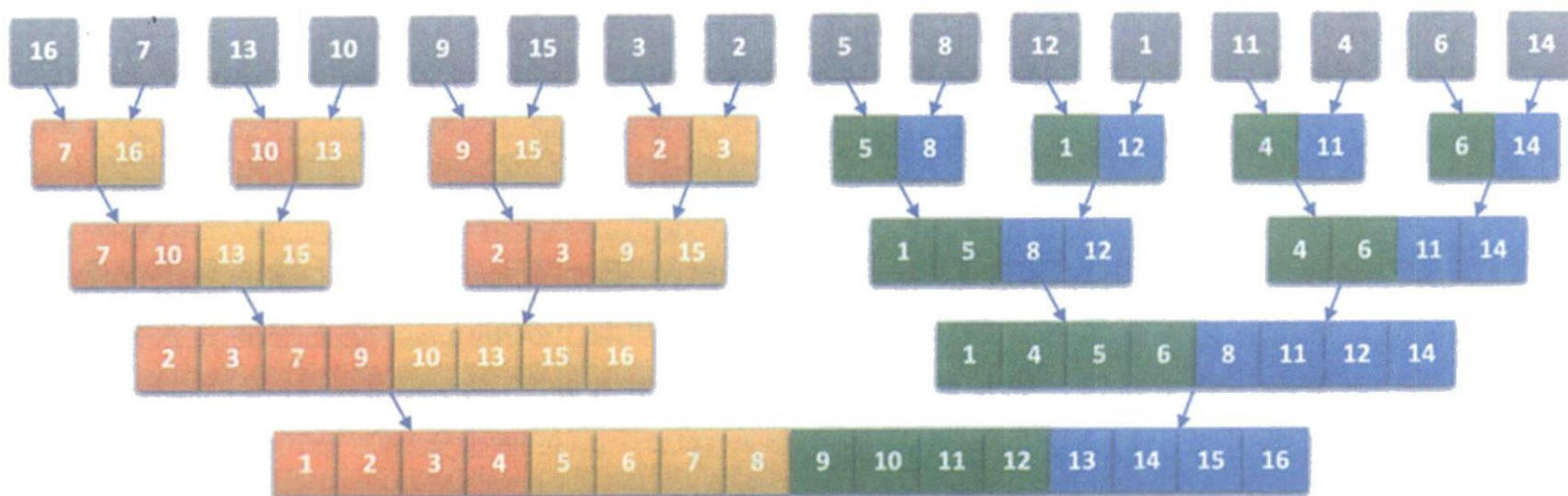
10.5 归并排序

归并排序的基本思想是：将两个（或以上）的有序表组成新的有序表。

更实际的意义：可以把一个长度为 n 的无序序列看成是 n 个长度为1的有序子序列，首先做两两归并，得到 $\lceil n/2 \rceil$ 个长度为2的有序子序列；再做两两归并， $\dots$ ，如此重复，直到最后得到一个长度为 n 的有序序列。

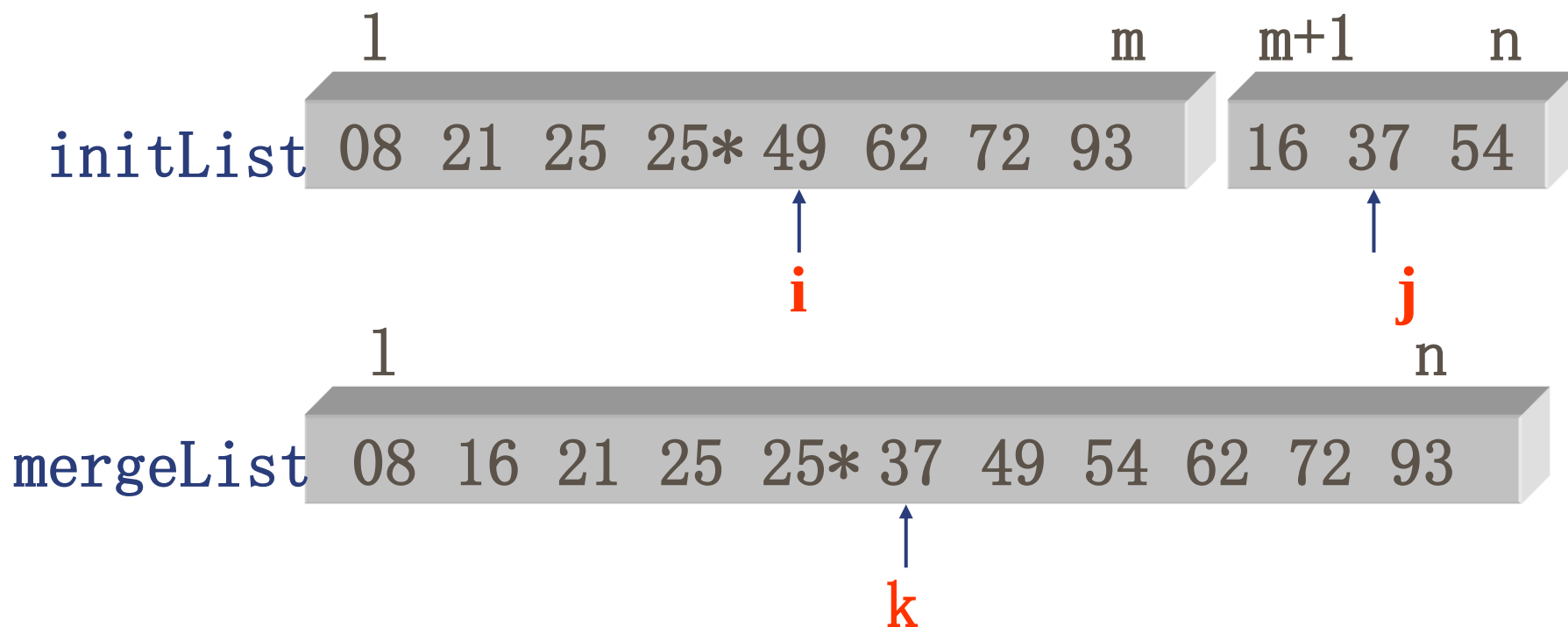
10.5 归并排序

{16, 7, 13, 10, 9, 15, 3, 2, 5, 8, 12, 1, 11, 4, 6, 14}



10.5 归并排序

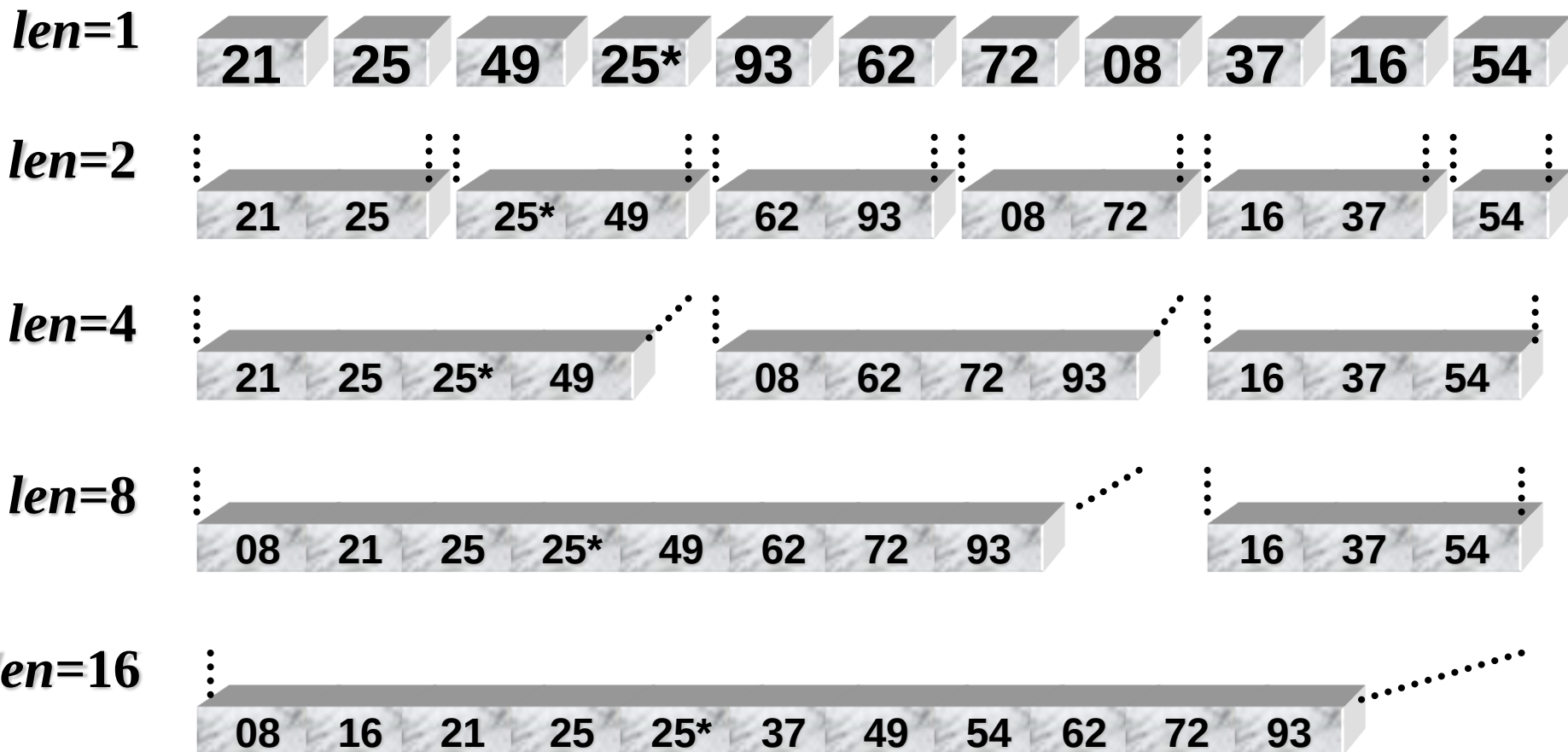
对象序列 **initList** 中有两个有序表 $V[1] \cdots V[m]$ 和 $V[m+1] \cdots V[n]$ 。它们可归并成一个有序表，存于另一对象序列 **mergedList** 的 $V[1] \cdots V[n]$ 中。



10.5 归并排序

例：关键字序列 $T = (21, 25, 49, 25^*, 93, 62, 72, 08, 37, 16, 54)$ ，请给出归并排序的具体实现过程。

10.5 归并排序



整个归并排序仅需 $\lceil \log_2 n \rceil$ 趟

10.5 归并排序

- 迭代的归并排序算法就是利用两路归并过程进行排序的。其基本思想是：
- 假设初始对象序列有 n 个对象，首先把它看成是 n 个长度为 1 的有序子序列 (归并项)，先做两两归并，得到 $\lceil n/2 \rceil$ 个长度为 2 的归并项 (如果 n 为奇数，则最后一个有序子序列的长度为 1)；再做两两归并， $\dots$ ，如此重复，最后得到一个长度为 n 的有序序列。
- 此算法的关键字比较次数和对象移动次数均为 $(m - l + 1) + (n - m) = n - l + 1$ 。

10.5 归并排序

```
void Merge(int S[], int T[],int i,int m,int n) {  
    /*i为第一个子表起始下标， m为第一个子表最后一个结点下标*/  
    int j,k; //k为合并表T的下标  
    for(j=m+1,k=i;i<=m&&j<=n;k++) {  
        if(S[i]<S[j]) T[k]=S[i++];  
        else T[k]=S[j++];  
    }  
    if(i<=m) { //将剩余的S[i..m]合并到T中  
        for(;i<=m;i++) T[k++]=S[i];  
    }  
    if(j<=n) { //将剩余的S[j..n]合并到T中  
        for(;j<=n;j++) T[k++]=S[j];  
    }  
}
```

10.5 归并排序

```
void MSort(int SR[], int TR1[], int s, int t){
    //将SR[s...t]归并到TR1[s...t]
    int TR2[MAXSIZE+1];
    if (s==t)
        TR1[s]=SR[s];
    else {
        int m=(s+t)/2; // 将SR[s..t]平分为 SR[s..m]和SR[m+1..t]
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t); // 将TR2[s..m]和TR2[m+1..t]归并到TR1[s..t]
    }
}

void MergeSort(SqList &L){
    MSort(L.r,L.r,1,L.length);
}
```

```
typedef struct seqlist
{
    int r[MAXSIZE];
    int length;
}SqList;
```

10.5 归并排序

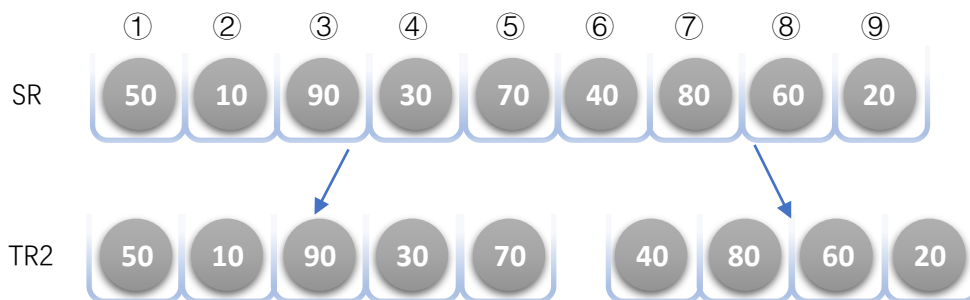
关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化

s=1, t=9, m=5



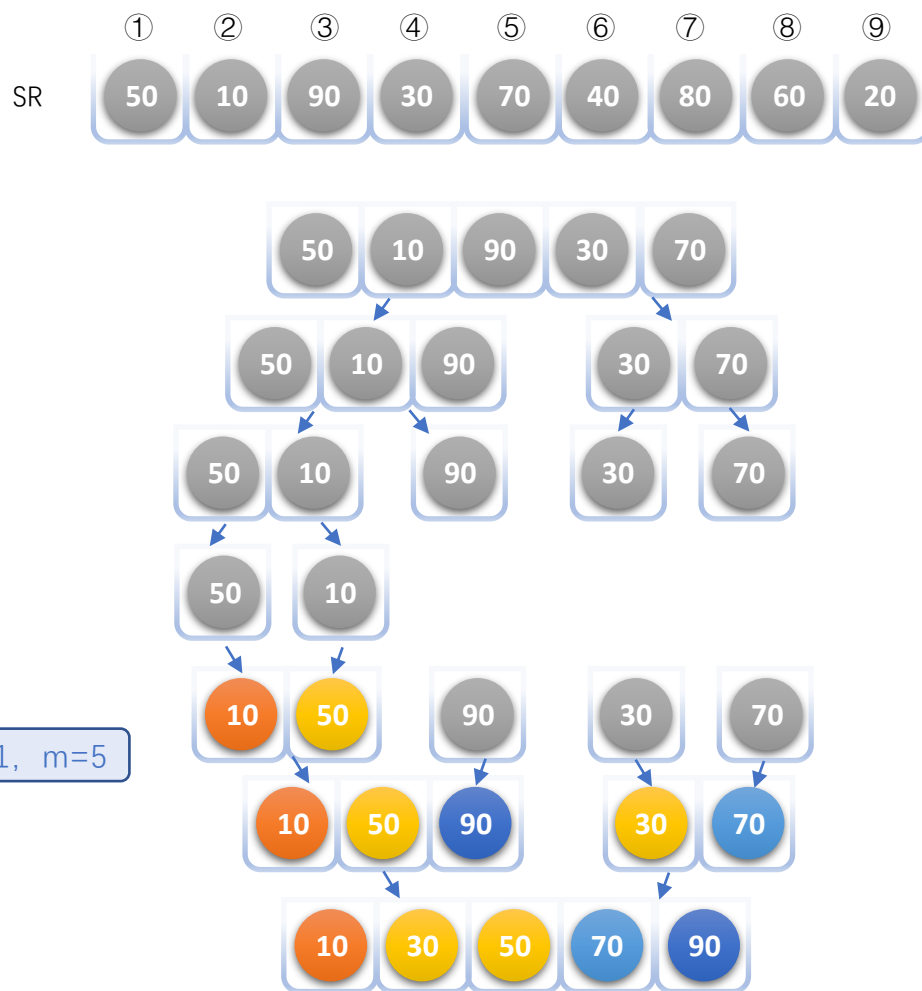
10.5 归并排序

关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化



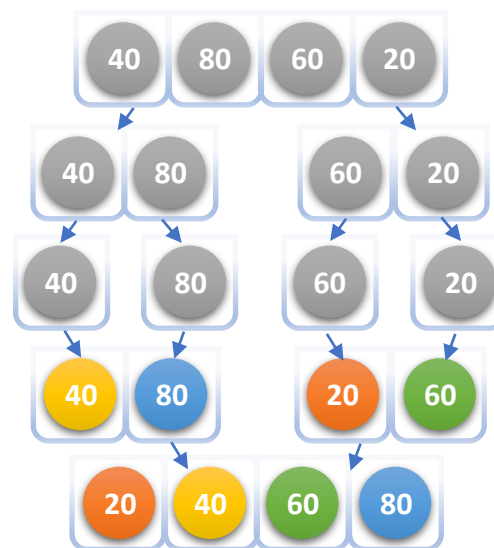
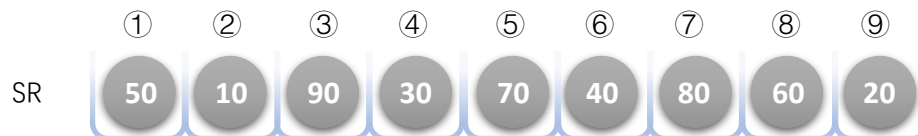
10.5 归并排序

关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化



m+1=6, t=9

10.5 归并排序

关键代码

```
void MergeSort(SqList &L)
{
    MSort(L.r,L.r,1,L.length);
}

void MSort(int SR[],int TR1[],int s, int t)
{
    int m;
    int TR2[MAXSIZE+1];
    if(s==t)
        TR1[s]=SR[s];
    else
    {
        m=(s+t)/2;
        MSort(SR,TR2,s,m);
        MSort(SR,TR2,m+1,t);
        Merge(TR2,TR1,s,m,t);
    }
}
```

数据变化



s=1, m=5, t=9

10.5 归并排序

关键代码

```
void Merge(int SR[],int TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=1, m=5, n=9



10.5 归并排序

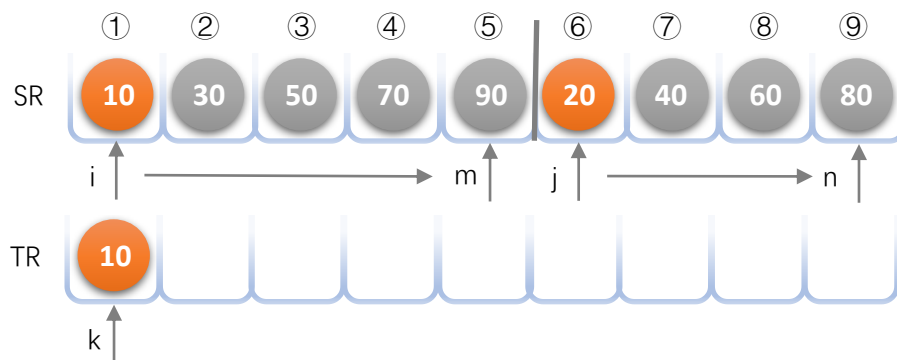
关键代码

```
void Merge(int SR[],int TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=1, m=5, n=9, j=6, k=1

SR[i]=10
SR[j]=20
TR[k]=10



10.5 归并排序

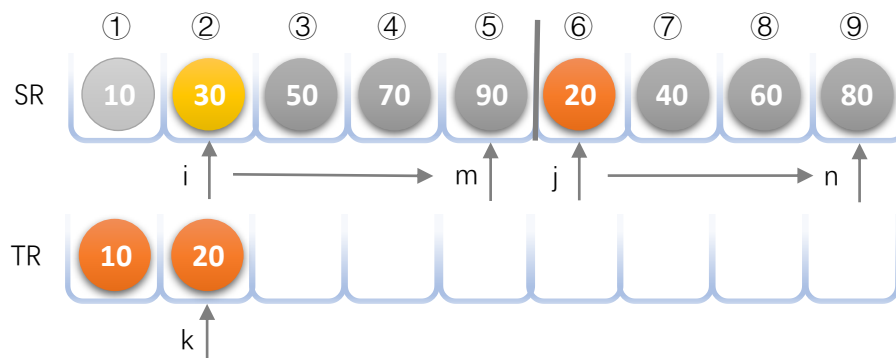
关键代码

```
void Merge(int SR[],int TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=2, m=5, n=9, j=6, k=2

SR[i]=30
SR[j]=20
TR[k]=20



10.5 归并排序

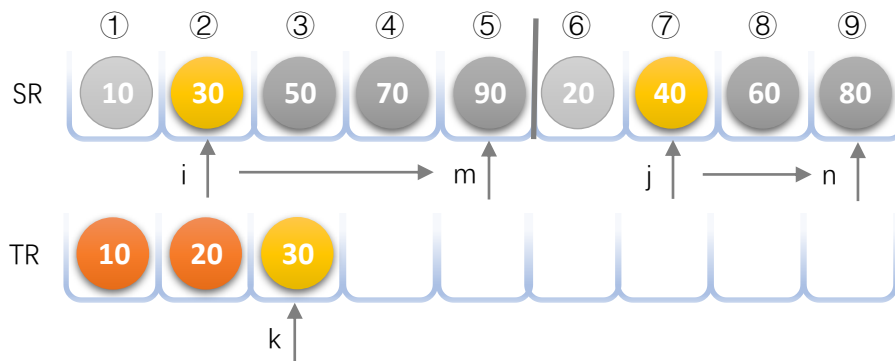
关键代码

```
void Merge(int SR[],int TR[],int i,int m,int n)
{
    int j,k,l;
    for(j=m+1,k=i;i<=m && j<=n;k++)
    {
        if (SR[i]<SR[j])
            TR[k]=SR[i++];
        else
            TR[k]=SR[j++];
    }
    if(i<=m)
    {
        for(l=0;l<=m-i;l++)
            TR[k+l]=SR[i+l];
    }
    if(j<=n)
    {
        for(l=0;l<=n-j;l++)
            TR[k+l]=SR[j+l];
    }
}
```

数据变化

i=2, m=5, n=9, j=7, k=3

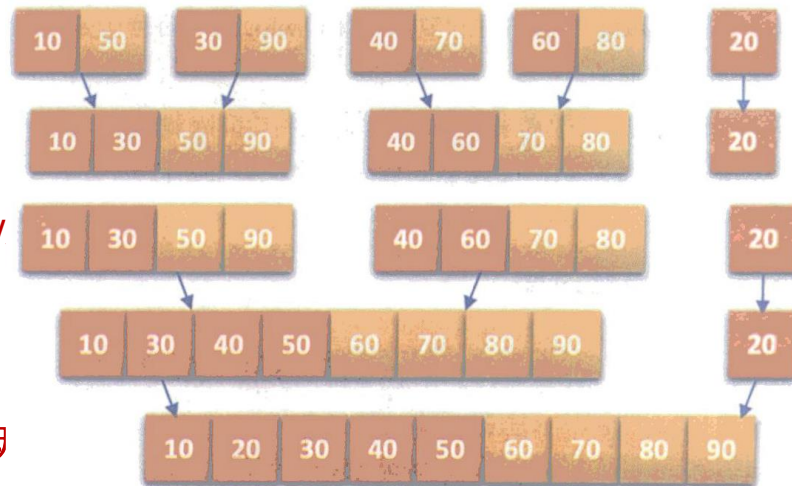
SR[i]=30
SR[j]=40
TR[k]=30



10.5 归并排序

归并排序非递归算法

```
void MergeSort2(SeqList &L) {  
    int k=1;  
    int *T=(int *)malloc(sizeof(int)L.length);  
    if(!T) {  
        printf("No enough memory!\n");  
        exit(-1);  
    }  
    while(k<L.length) {  
        MergePass(L.r,T,k,L.length); //  
        k=k*2; //子序列加倍  
        MergePass(T,L.r,k,L.length);  
        k=k*2; //数将T中已经两  
    }  
}
```



10.5 归并排序

```
void MergePass(int S[],int T[],int s,int n) {
```

```
    //将S[]中长度为s的子序列两两归并到T[]
```

```
    int i=1, j;
```

```
    while(i<=n-2*s+1){ //n-(n-2s+1)+1=2s 说明下标i后面有两个子表, 长度为s
```

```
        Merge(S,T,i,i+s-1,i+2*s-1); //两两归并
```

```
        i=i+2*s;
```

```
    }
```

```
    if(i<n-s+1) //归并最后两个序列
```

// $n-(n-s+1)+1=s$, 由于i小于n-s+1, 所以包括i结点本身至少s+1个结点, 有两个子表, 第二个子表长度小于s

```
        Merge(S,T,i,i+s-1,n);
```

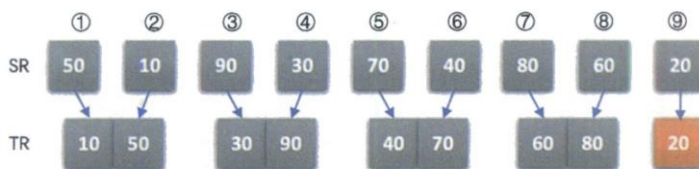
```
    else { //若最后只剩下单个子序列
```

```
        for(j=i;j<=n;j++)
```

```
            T[j]=S[j];
```

```
    }
```

```
}
```



10.5 归并排序

算法分析:

- 时间效率: $O(n\log_2 n)$

因为在递归的归并排序算法中，函数Merge()做一趟两路归并排序，需要调用merge()函数 $\lceil n/(2h) \rceil \approx O(n/h)$ 次，而每次merge()要执行比较 $O(h)$ 次，另外整个归并过程有 $\lceil \log_2 n \rceil$ “层”，所以算法总的时间复杂度为 $O(n\log_2 n)$ 。

10.5 归并排序

算法分析：

- 时间效率： $O(n\log_2 n)$

空间效率： $O(n)$

因为需要一个与原始序列同样大小的辅助序列（TR）。这正是此算法的缺点。

- 稳定性： 稳定

正在答疑
